

React When You Need To: Event-Triggered Asynchronous Inference for VLA Policies

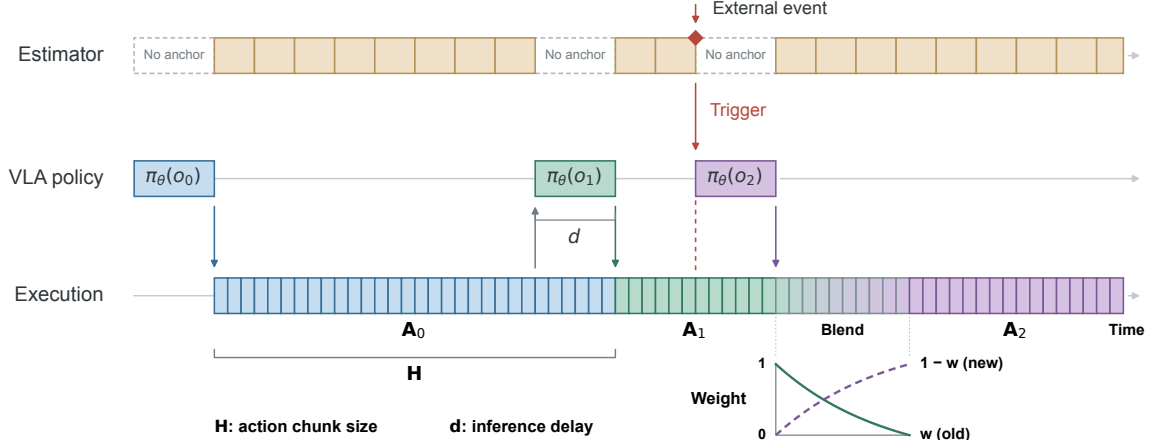


Fig. 1: Overview of the proposed event-triggered dynamic inference strategy. The event estimator continuously measures scene changes relative to the observation used for the most recent VLA inference. Without a significant event, the scheduler retains the maximum feasible inference gap to preserve motion consistency. Upon event detection, it triggers early inference and smoothly transitions to the updated action chunk for prompt reactivity.

Abstract—Vision-Language-Action (VLA) models commonly predict action chunks, limiting their ability to react to environmental changes during execution. Existing asynchronous inference methods improve reactivity but typically rely on a fixed inference gap. In this paper, we propose an event-guided dynamic inference strategy that adapts the inference gap according to scene changes observed since the previous inference. Thereby, it simultaneously preserves motion consistency and prompt reactivity. Across static and dynamic real-world settings, our method consistently performs best, averaging 95% success and exceeding the strongest baseline by 55 percentage points. The code will be made publicly available upon acceptance. The project page is available at [this URL](#).

Keywords: Vision-Language-Action Models, Asynchronous Inference, Event-Triggered Inference Scheduling

I. INTRODUCTION

Vision-Language-Action (VLA) models have shown strong generalization across robot manipulation tasks [1]–[6]. Existing VLA models commonly predict action chunks, generating multiple future actions per inference [7]. Under conventional synchronous execution, the robot completes the current chunk before requesting the next one. Consequently, when unexpected environmental changes occur during chunk execution, the robot continues to follow the precomputed action chunk and cannot react until the next policy inference.

Although primarily developed to eliminate inter-chunk stalls, asynchronous execution also partially mitigates this delayed response [8]–[14]. It initiates the next policy inference while the robot continues executing the ongoing action chunk. It enables more frequent policy updates, thereby reducing the response latency to environmental changes.

However, existing asynchronous execution methods typically use a fixed inference gap, creating an inherent trade-off between reactivity and execution reliability. Shortening the inference gap enables more frequent policy inference and faster reactions to environmental changes. However, it also necessitates more frequent merging of consecutive action chunks, which may introduce trajectory inconsistencies and compromise execution reliability [15], [16]. In contrast, a longer inference gap better preserves motion continuity but delays the robot’s response to environmental changes [8].

To address this limitation, we propose an event-triggered dynamic inference strategy that schedules VLA inference according to environmental changes. During execution, an event estimator measures the degree of change between the current scene and the anchor scene observed during the most recent policy. The resulting event signal indicates whether early policy inference is required. While maintaining stall-free execution, our strategy retains the largest feasible inference gap when no significant event is detected. Once an event is detected, it immediately triggers a new inference without waiting until the default gap. In this way, our strategy preserves the motion consistency of long inference gaps while retaining the prompt reactivity of short ones.

The main contributions are summarized as follows:

- It maintains the largest feasible gap in the absence of significant events and triggers early inference upon event detection, thereby simultaneously preserving motion consistency and prompt reactivity.
- We develop a lightweight event estimator based on the VLA vision encoder to quantify scene changes since

the most recent policy inference. Our event estimation is interleaved with VLA inference and operates at up to 15 Hz in our setup.

- We conduct comprehensive real-world experiments involving both static and dynamic tasks. Our method achieves an average full-task success rate of 95% across the three evaluation settings, representing an absolute improvement of 55% over the best competing methods.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-Language-Action (VLA) models [1]–[6] leverage large-scale vision-language pretraining to learn end-to-end policies that map visual observations and language instructions to low-level robot actions. Recent advances have followed several complementary architectural directions. Autoregressive models [5], [6], [17] formulate robot control as next-token prediction over discretized action tokens, naturally extending the training paradigm of vision-language models to action generation. Diffusion-based approaches [4], [18]–[21] instead model continuous action distributions, enabling smooth and expressive behaviors at the cost of iterative inference. Flow-based methods, exemplified by π_0 [22], replace diffusion sampling with conditional flow matching to improve inference efficiency. Meanwhile, dual-system architectures such as $\pi_{0.5}$ [23] and GR00T-N1 [24] decouple high-level semantic reasoning from low-level action generation to support long-horizon manipulation. Nevertheless, VLA models are commonly deployed with synchronous inference, where action generation and robot execution proceed sequentially. This execution strategy introduces pronounced stalls between consecutive action chunks and limits the policy’s ability to respond to environmental changes during chunk execution [8].

B. Asynchronous Inference

To eliminate this stop-and-go behavior, asynchronous inference has received increasing research attention [8]–[14]. It generates the next action chunk while the current one is still being executed, ensuring continuous action availability without waiting for model inference. SmolVLA [9] adopts such an asynchronous execution scheme, but directly switching between independently generated chunks can introduce inter-chunk discontinuities. RTC [8] mitigates this issue by freezing the action prefix guaranteed to be executed during inference and inpainting the remaining actions, while softly incorporating additional overlapping actions to improve continuity. However, its gradient-based inference-time guidance introduces additional computational overhead. Training-time RTC [12] addresses this limitation by simulating inference delays during training and directly learning to generate an action postfix conditioned on the committed prefix, thereby eliminating the additional inference-time guidance. Other studies address the temporal misalignment between prediction and execution through execution-time state condition-

ing [11], outdated-action filtering [13], per-step action correction [10], or trajectory refitting [14]. Despite these advances, existing asynchronous methods typically invoke the policy continuously or at a predefined rate, without adapting the inference schedule to environmental changes.

III. PROBLEM FORMULATION

Following the definition in [8], an action chunking policy is denoted by $\pi(\mathbf{A}_t|\mathbf{o}_t)$, where $\mathbf{A}_t = [\mathbf{a}_t, \mathbf{a}_{t+1}, \dots, \mathbf{a}_{t+H-1}]$ is a chunk of future actions, $\mathbf{o}_t$ is an observation, t indicates a controller timestep, and H denotes the *prediction horizon*. Let Δt be the sampling period of the robot controller, and let δ be the continuous time required for the policy to generate an action chunk. The *inference delay* is defined as $d := \lceil \delta / \Delta t \rceil$. The execution horizon s denotes the number of actions actually executed from each chunk.

On this basis, let ΔT denote the elapsed time between the completion of one inference call and the initiation of the next. We define the corresponding number of controller steps as the *inference gap* g :

$$g := \left\lfloor \frac{\Delta T}{\Delta t} \right\rfloor. \quad (1)$$

The robot can operate continuously without inter-chunk execution stalls, as long as the system parameters satisfy:

$$g \leq H - d. \quad (2)$$

In practice, selecting an appropriate inference gap g presents a fundamental trade-off. A small gap enables frequent policy updates and improves responsiveness, but also leads to frequent transitions between action chunks, which may introduce action instable execution [15], [16]. Conversely, a large gap reduces the frequency of chunk transitions but makes the policy less responsive to environmental changes occurring during chunk execution [8].

This trade-off motivates our central problem: *can the inference gap be dynamically adapted to environmental changes while preserving seamless execution?* Specifically, the system is expected to maintain a larger inference gap when little environmental change has occurred since the current action chunk was predicted, and automatically switch to a small gap when a significant change is detected, enabling the policy to respond with minimal delay.

IV. METHOD

To address this problem, we propose an event-triggered dynamic inference framework that uses a data-driven event estimator to monitor whether the currently executing action chunk remains applicable to the evolving scene and schedules policy inference accordingly.

A. Event-Supervision Construction from Human Demonstrations

Rather than manually annotating the occurrence and relevance of such events, we automatically derive event supervision from human demonstrations. The entire process consists of three steps:

1) *Data Collection*: For each task, we collect 40 demonstrations under static environmental conditions and 160 demonstrations under dynamic environmental conditions via teleoperation, forming $\mathcal{D}_{\text{static}}$ and $\mathcal{D}_{\text{dynamic}}$, respectively. The complete demonstration dataset is formed by combining the two:

$$\mathcal{D} = \mathcal{D}_{\text{static}} \cup \mathcal{D}_{\text{dynamic}}. \quad (3)$$

The dataset comprises N demonstrations:

$$\mathcal{D} = \left\{ \tau^{(i)} \right\}_{i=1}^N, \quad (4)$$

where each demonstration $\tau^{(i)}$ is defined as:

$$\tau^{(i)} = \left(\ell^{(i)}, \{(\mathbf{I}_t^{(i)}, \mathbf{s}_t^{(i)}, \mathbf{a}_t^{(i)})\}_{t=1}^{T_i} \right). \quad (5)$$

Here, $i \in \{1, \dots, N\}$ indexes the demonstration, and T_i denotes its trajectory length. Specifically, $\ell^{(i)}$ represents the language instruction, while $\mathbf{I}_t^{(i)}$, $\mathbf{s}_t^{(i)}$, and $\mathbf{a}_t^{(i)}$ denote the visual image, robot proprioceptive state, and action at time step t , respectively.

2) *VLA Fine-Tuning*: We fine-tune the pretrained $\pi_{0.5}$ VLA policy on $\mathcal{D}_{\text{static}}$ using LoRA [25] to establish a reference model of nominal task behavior. The dynamic subset is excluded because actions recorded after an abrupt event reflect the demonstrator’s adaptation to environmental information unavailable in the initial observation. Including such actions would not only violate the causal alignment between the initial observation and its supervised action chunk, but also incorporate event-induced responses into the nominal reference.

After fine-tuning, all VLA parameters are frozen. The resulting reference policy is subsequently applied offline to the complete demonstration dataset to generate nominal action chunks for event-supervision construction.

Algorithm 1 Dataset Construction for Event Estimator

Require: VLA model π , Dataset $\mathcal{D}$, vision encoder E_{vis} , horizon H , and inference latency d

```

1: Initialize  $\mathcal{D}_{\text{event}} \leftarrow \emptyset$ 
2: for  $i = 1$  to  $N$  do
3:   for  $j = 1$  to  $T_i - H + 1$  do
4:      $\mathbf{A}_j^{(i)} \leftarrow [\mathbf{a}_j^{(i)}, \dots, \mathbf{a}_{j+H-1}^{(i)}]$   $\triangleright$  demo actions
5:      $\hat{\mathbf{A}}_j^{(i)} \leftarrow \pi(\ell^{(i)}, \mathbf{I}_j^{(i)}, \mathbf{s}_j^{(i)})$   $\triangleright$  predicted actions
6:      $\mathbf{z}_{\text{anchor}}^{(i)} \leftarrow E_{\text{vis}}(\mathbf{I}_j^{(i)})$   $\triangleright$  anchor visual feature
7:     for  $k = 1$  to  $H - d$  do
8:        $\mathbf{z}_{j+k}^{(i)} \leftarrow E_{\text{vis}}(\mathbf{I}_{j+k}^{(i)})$   $\triangleright$  current visual feature
9:        $e_{j,j+k}^{(i)} \leftarrow \text{MSE}(\mathbf{A}_j^{(i)}[k:], \hat{\mathbf{A}}_j^{(i)}[k:])$ 
10:       $\mathbf{p} \leftarrow (\mathbf{z}_{\text{anchor}}^{(i)}, \mathbf{z}_{j+k}^{(i)}, e_{j,j+k}^{(i)})$   $\triangleright$  event pair
11:       $\mathcal{D}_{\text{event}} \leftarrow \mathcal{D}_{\text{event}} \cup \{\mathbf{p}\}$ 
12:    end for
13:  end for
14: end for
15: return  $\mathcal{D}_{\text{event}}$ 

```

3) *Event Estimator Dataset Construction*: Afterwards, we leverage the fine-tuned VLA policy to perform offline inference across the demonstration dataset $\mathcal{D}$. As detailed in Algorithm 1, for each inference performed, we record the corresponding visual representation as the anchor visual feature $\mathbf{z}_{\text{anchor}}^{(i)}$. We then compare the predicted chunk with the corresponding demonstrated actions, whose divergence indicates whether an event occurring during chunk execution has altered the intended behavior. Specifically, we define the event score $e_{j,j+k}$ as a continuous proxy for the likelihood that an abrupt event has occurred during chunk execution and altered the intended motion (Line 9 in Algorithm 1). The first subscript of event score denotes the anchor inference timestep, and the second identifies the current time step evaluated against that anchor. Collecting the labeled anchor-current feature pairs yields the event estimator dataset $\mathcal{D}_{\text{event}}$.

B. Event Estimator

1) *Event Score Estimation*: Based on the event dataset $\mathcal{D}_{\text{event}}$ constructed above, we develop a lightweight event estimator that predicts the event score by comparing the visual observation captured at the most recent policy inference with the current observation.

Specifically, our event estimator consists of a frozen VLA vision encoder E_{vis} and a lightweight Transformer-based event decoder D_{event} . Let t_0 denote the time step of the most recent policy inference (anchor) and t denote the current time step ($t > t_0$). The vision encoder extracts visual representations $\mathbf{z}_{t_0}$ and $\mathbf{z}_t$ from the respective observations $\mathbf{I}_{t_0}$ and $\mathbf{I}_t$.

$$\begin{aligned} \mathbf{z}_{t_0} &= E_{\text{vis}}(\mathbf{I}_{t_0}), \\ \mathbf{z}_t &= E_{\text{vis}}(\mathbf{I}_t). \end{aligned} \quad (6)$$

The transformer-based event decoder then compares the resulting visual representations and predicts the corresponding event score $e_{t_0,t}$:

$$e_{t_0,t} = D_{\text{event}}(\mathbf{z}_{t_0}, \mathbf{z}_t), \quad (7)$$

where the first subscript of event score denotes the anchor scene, and the second denotes the current scene compared against it.

Reusing the pretrained VLA visual encoder instead of introducing another backbone offers two key advantages: (i) *Task-Aligned Representations*: Pretrained on manipulation scenes to support action generation, the encoder inherently preserves task-critical spatial and semantic information. (ii) *Computational Efficiency*: The anchor feature $\mathbf{z}_{t_0}$ is already computed during policy inference and can be cached at zero marginal cost. Furthermore, extracting $\mathbf{z}_t$ and evaluating D_{event} run within the interval between policy inferences, avoiding introducing computational latency to action generation.

2) *Score-to-Event Mapping*: To convert the continuous event score into a binary inference decision, we define a P90 threshold θ_{P90} as the 90th percentile of the event-score distribution obtained from static demonstrations. As

illustrated in Fig. 2, although the event-score distributions of dynamic demonstrations vary substantially across tasks, the static distributions exhibit similar shapes and consistently concentrate in the low-score region. Scores from static demonstrations primarily characterize variations that should not trigger an early policy inference. We therefore treat the lower 90% of the static distribution as non-event variation. To suppress triggers caused by transient score fluctuations, an event is declared only when the estimated score exceeds θ_{P90} in two consecutive evaluations, upon which a new policy inference is triggered.

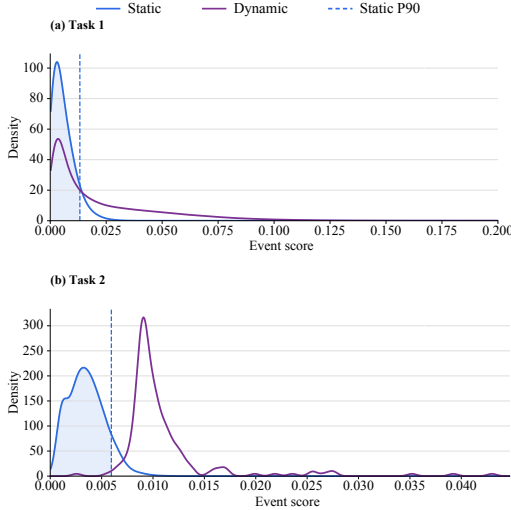


Fig. 2: Representative event-score distributions under static and dynamic conditions for Task 1 and Task 2 detailed in Sec. V-A2. The dashed blue line marks the task-specific 90th percentile of the static distribution, used as the event threshold θ_{P90} .

C. Event-Driven Inference Scheduling

As illustrated in Fig. 1, policy inference and event estimation run alternately without temporal overlap, thereby avoiding competition for computational resources. Each rollout begins with an initial policy inference that generates an action chunk from the current observation. The visual feature extracted from the inference-time scene image by the VLA vision encoder is retained as the anchor feature for subsequent event estimation. Once policy inference is complete, the event estimator runs repeatedly, comparing the latest available observation with the anchor to determine whether an event requiring a new policy inference has occurred.

If no event is detected, the next policy inference is automatically initiated after $H - d$ actions of the current chunk have been executed, leaving the final d actions to cover the inference delay and ensure seamless execution. If an event is detected before this point, the system instead triggers the next inference immediately using the latest observation.

For this event-triggered early inference, the new action chunk becomes available before the ongoing chunk has been

fully executed. To ensure a smooth transition, we blend the overlapping actions as follows:

For an overlap of $N \geq 2$ actions, the blending weights are defined as

$$\mathbf{w} = \frac{\exp\left(-\beta \frac{\mathbf{k}}{N-1}\right) - \exp(-\beta)\mathbf{1}_N}{1 - \exp(-\beta)}, \quad (8)$$

where $\mathbf{k} = [0, 1, \dots, N-1]^\top$ indexes the actions in the overlapping area, N is the overlap length, $\mathbf{1}_N \in \mathbb{R}^N$ is the all-ones vector, and $\beta > 0$ controls the decay of the weights. The exponential is applied element-wise. The weight assigned to the previous chunk decreases from 1 to 0.

The overlapping actions are then merged as:

$$\mathbf{A}_{\text{merge}} = \mathbf{w} \odot \mathbf{A}_{\text{old}} + (\mathbf{1} - \mathbf{w}) \odot \mathbf{A}_{\text{new}}, \quad (9)$$

where $\mathbf{A}_{\text{old}}$ and $\mathbf{A}_{\text{new}}$ denote the overlapping segments of the previous and new chunks, respectively.

V. EXPERIMENT

In this section, we conduct two groups of real-world manipulation experiments to address the following research questions:

- **Q1:** Can the proposed method maintain reliable task execution in static environments?
- **Q2:** Can the proposed method respond promptly and effectively to environmental changes through event-triggered inference scheduling?
- **Q3:** Can our method achieve consistently high task performance across static and dynamic settings?

A. Experimental Setup

TABLE I: Configuration Details

Setting	Value
VLA fine-tune steps	30,000
VLA fine-tune batch size	32
Action-chunk horizon H	50
Blending weight decay β	1.2
Demonstration sampling rate	30 Hz
Execution frequency	30 Hz
Event estimator frequency	15 Hz
Inference delay d	5
RTC inference delay d_{RTC}	8
RTC inference gap g	20
DynamicVLA inference gap g	0

[†] The inference delay is $d = 5$ action steps for all methods except RTC. RTC uses an inference delay of $d_{\text{RTC}} = 8$ action steps, corresponding to an execution horizon of $s = 28$ in the RTC formulation.

1) *Robotic Platform:* The experimental platform consists of a Franka Emika Panda robot and three Intel RealSense D435i cameras. The robot controller runs on a NUC with an Intel i7-10700 CPU, while the VLA model and the proposed event estimator are executed on a separate workstation equipped with an NVIDIA RTX 3090 GPU. We fine-tune the VLA model using six NVIDIA A40 GPUs. Table I summarizes the concrete training and inference configurations

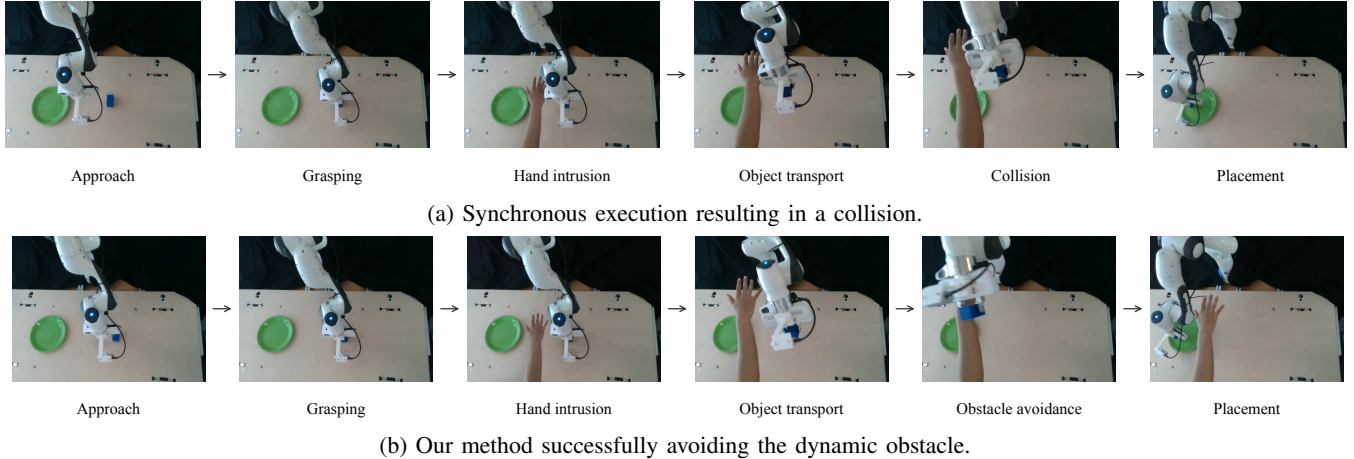


Fig. 3: Qualitative comparison of synchronous execution and our method under repeated obstructions caused by a moving human hand during a pick-and-place task.

2) *Evaluation Tasks* : We evaluate the proposed method on two manipulation tasks involving distinct types of environmental changes.

- **Task 1: Pick-and-place with static and dynamic obstacles** (Fig. 3b). The robot is instructed to pick up a designated blue cube and place it on a green plate while avoiding an obstacle in the transport region. The obstacle is introduced by a human operator extending a hand into the workspace. We evaluate two conditions: (i) In the static condition, the obstacle is introduced before transportation and remains stationary, while its location is varied across trials. (ii) In the dynamic condition, we adopt a closed-loop obstruction protocol. During transport, the operator introduces the hand into the robot’s current motion path without intentionally making contact. Whenever the robot changes its path to avoid the hand, the operator repositions it to obstruct the updated path. This process continues until the robot reaches the placement region.
- **Task 2: Placement onto a moving target** (Fig. 4). The robot starts with an object already grasped and is instructed to place it on a movable plate. We adopt a closed-loop target-relocation protocol in which the operator repeatedly moves the plate during execution. Whenever the grasped object approaches the region above the plate, the operator moves the plate away from the robot’s current placement trajectory, varying both its position and direction of motion. This process continues until the robot successfully places the object or the trial terminates. Consequently, successful execution requires the policy to repeatedly update its ongoing action sequence based on recent observations and track the relocated target.

B. Comparing Methods

We compare our method against three representative execution baselines and four fixed-gap ablations. All configurations use the same $\pi_{0.5}$ policy backbone [23].

- **Synchronous**: Policy inference and action execution alternate without overlap.
- **DynamicVLA**: DynamicVLA [13] improves responsiveness in dynamic environments by discarding actions whose intended execution steps elapse during inference and updating the execution buffer with the remaining chunk. For a fair comparison, we replace its original VLA backbone with $\pi_{0.5}$ while retaining this update strategy.
- **RTC**: RTC [8] uses inference-time inpainting and soft masking to condition new action chunks on previously committed actions.
- **Ours**: We apply the proposed method described in Section IV to the shared backbone.
- **Fixed-gap triggering ablations**: We disable the event estimator and use four fixed inference gaps, $g \in \{0, 5, 25, 45\}$, to schedule subsequent inferences. Each variant is denoted as Ablation ($g = x$), where x is the corresponding fixed gap. All other components and execution settings remain unchanged.

C. Experimental Procedure

Each method is evaluated in 20 full-task trials under each of the three settings: Task 1 with a static obstacle, Task 1 with a dynamic obstacle, and Task 2 with a moving target. Since Task 1 comprises three sequential stages, a failure at an earlier stage would otherwise prevent the evaluation of the remaining stages. We manually restore the required intermediate state and resume execution, ensuring that each stage is evaluated over 20 attempts. Such recovery contributes only to the stage-wise evaluation, while the original full-task trial remains recorded as a failure.

D. Evaluation Metrics

For *Task 1*, under both static and dynamic obstacle conditions, performance is evaluated at the stage and full-task levels. A pick is successful when the robot grasps the designated cube; obstacle avoidance is successful when the

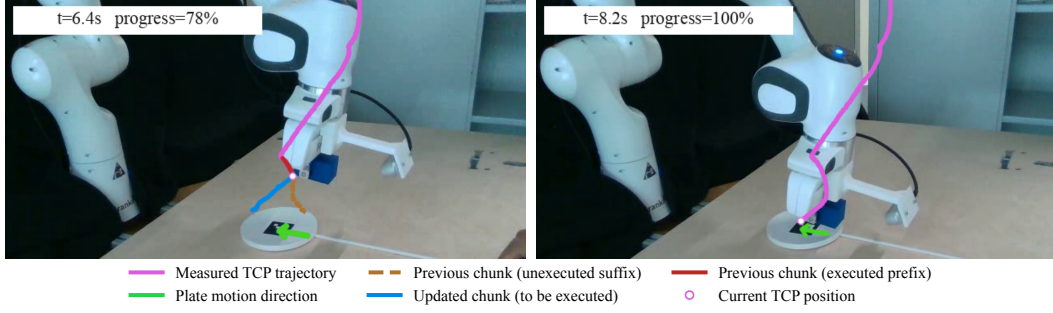


Fig. 4: Representative Task 2 execution under target-motion perturbation, showing the action-chunk update at $t = 6.4$ s and the complete TCP trajectory at task completion ($t = 8.2$ s).

robot transports the grasped cube to the placement region without contacting the obstacle; and placement is successful when the robot releases the cube onto the target plate. A full-task success requires all three stages to be completed successfully within a single trial. *Task 2* evaluates only the placement on moving target and adopts the same success criterion, requiring the grasped object to be released onto the moving plate.

Based on the above criteria, the success rate for each stage or full task is computed as:

$$SR_c = \frac{N_c^{\text{success}}}{N_c^{\text{trials}}} \times 100\%, \quad (10)$$

where c denotes the evaluated stage or full task, and N_c^{success} and N_c^{trials} denote the corresponding numbers of successful and total trials, respectively.

VI. RESULTS AND DISCUSSION

A. Performance in Static Environments (Q1)

Table IIa reports the phase-wise and full-task success rates of the evaluated execution methods in the pick-and-place task with static obstacles. All methods performed well in the obstacle-avoidance and placement phases, achieving 100% success in both phases, except for DynamicVLA that achieved a placement success rate of 90%. The performance variance primarily concentrated in the picking phase. Synchronous execution, the fixed-gap ablations ($g = 25$ and $g = 45$), and our method achieved 100% picking success, whereas the ablation with $g = 5$ achieved only 30%. By contrast, DynamicVLA, RTC, and the ablation with $g = 0$ failed in all picking trials. Compared with the other two phases, picking places higher demands on precise gripper-object alignment. Inspection of the failed trials showed that most failures occurred when the gripper tip pressed against the object during grasping, displacing it before a stable grasp could be established.

The fixed-gap ablation tests reveal a clear trend: picking success increased markedly as the inference gap increased. In static environments, short inference gaps lead to frequent action-chunk merging, which may disrupt the fine-grained motions required for grasping. In contrast, longer gaps allow longer action sequences to be executed without interruption,

TABLE II: Stage-wise and full-task success rates for Task 1.

(a) Success Rate with Static Obstacle [%]

Mode	Pick	Obstacle Avoidance	Place	Full-Task
Synchronous	100	100	100	100
DynamicVLA	0	100	90	0
RTC	0	100	100	0
Ablation ($g = 0$)	0	100	100	0
Ablation ($g = 5$)	30	100	100	30
Ablation ($g = 25$)	100	100	100	100
Ablation ($g = 45$)	100	100	100	100
Ours	100	100	100	100

(b) Success Rate with Dynamic Obstacle [%]

Mode	Pick	Obstacle Avoidance	Place	Full-Task
Synchronous	100	0	100	0
DynamicVLA	0	0	100	0
RTC	15	95	100	15
Ablation ($g = 0$)	0	95	100	0
Ablation ($g = 5$)	15	90	100	15
Ablation ($g = 25$)	90	0	95	0
Ablation ($g = 45$)	100	0	100	0
Ours	100	100	100	100

† All stage metrics report success rates across 20 trials, with manual resets upon failure in prior stages. Full-Task indicates end-to-end success without intervention.

thereby preserving motion consistency and improving grasping reliability. By maintaining a long inference gap when no significant environmental change is detected, our method achieves a 100% picking success rate, matching the fixed long-gap ablations.

B. Performance in Dynamic Environments (Q2)

1) *Pick-and-Place with Dynamic Obstacle*: As shown in Table IIb, interaction with the moving obstacle occurs primarily during the transport phase after the object has been grasped. Consequently, the results for the picking and placement phases remain broadly consistent with those observed in the static setting. During dynamic obstacle avoidance, our method again achieves the highest success rate of 100%. However, the baseline methods exhibit a trend almost opposite to that observed in the static setting. Methods with frequent policy updates, including RTC and the short-gap ablations ($g = 0$ and $g = 5$), perform strongly in dynamic obstacle avoidance, whereas synchronous execution and the

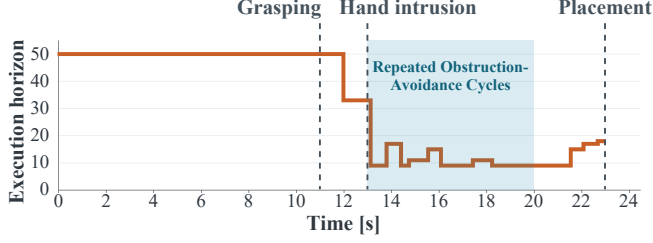


Fig. 5: Execution horizon during Task 1. Dashed lines mark task events. The shaded region indicates repeated obstruction-avoidance cycles.

long-gap ablations ($g = 25$ and $g = 45$) fail to react to the moving obstacle.

An illustrative comparison is shown in Fig. 3. Upon a sudden hand intrusion, our event estimator promptly detects the environmental change and triggers an early policy inference. As the person repeatedly moves the hand to block the robot, the resulting event triggers maintain a high inference frequency, allowing the robot to continuously update its action plan and maneuver around the hand. Figure 5 visualizes the number of actions executed from each predicted action chunk during this process. Following the hand intrusion, the executed chunk lengths decrease substantially, directly reflecting the repeated early-inference triggers required to respond to the continuously changing obstacle. In contrast, synchronous execution cannot respond to unexpected changes that occur during the execution of an action chunk. It therefore continues executing the previously predicted actions and collides with the hand.

2) *Place onto a Moving Target*: Unlike the preceding task, this task contains no picking phase and directly evaluates whether the robot can adapt its placement motion to a continuously moving target. As shown in Fig. 6, synchronous execution and the fixed-gap ablation ($g = 45$) achieve success rates of only 5% and 15%, respectively. These results indicate that infrequent policy updates provide insufficient responsiveness to the target motion.

Meanwhile, naively reducing the inference gap is not sufficient. The ablation with $g = 0$ achieves a success rate of only 30%, substantially lower than the 60% achieved with $g = 5$. By dynamically adapting the inference gap to

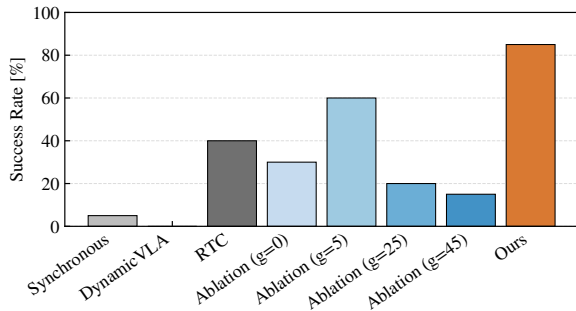


Fig. 6: Success rates for Task 2 (*Place onto a Moving Target*) across baseline methods, fixed-gap ablations, and our method.

the observed target motion, our method achieves the highest success rate of 85%, outperforming RTC by 45 percentage points and the best fixed-gap ablation by 25 percentage points.

Fig. 4 illustrates a representative execution of our method during moving-target placement. At $t = 6.2$ s, the plate is pushed toward the robot base, and the event estimator detects this change and triggers a new round of policy inference. The updated action chunk becomes available and begins execution at $t = 6.4$ s, resulting in the shortened execution horizon shown in Fig. 7. As visualized in Fig. 4, the updated chunk, shown in blue, redirects the robot toward the new plate position, whereas the unexecuted suffix of the previous chunk, shown in orange, continues toward the outdated target position. This example demonstrates how adaptive inference timing replaces outdated actions when needed, enabling prompt responses without relying on continuously high-frequency inference.

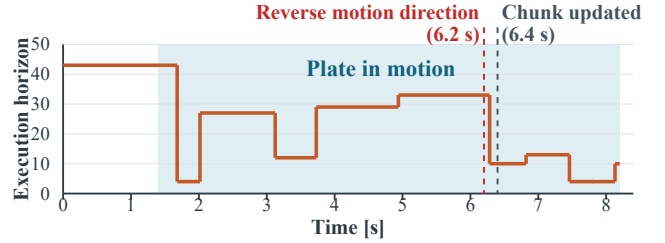


Fig. 7: Execution horizon during Task 2 under target perturbation. The shaded region indicates the plate in motion; dashed lines mark motion reversal and the subsequent chunk update.

C. Overall Performance Across Evaluation Settings

To evaluate the overall robustness of the execution methods, Fig. 8 aggregates the full-task success rates across the three evaluation settings: Task 1 with a static obstacle (full-horizon), Task 1 with a dynamic obstacle (full-horizon), and Task 2. We compute the overall performance as the sum of the full-task success rates across the three equally weighted evaluation settings.

The baseline methods exhibit clear scenario-dependent trade-offs. Synchronous execution and the long-gap ablations perform reliably in the static setting but fail to respond effectively to dynamic changes. Conversely, RTC and the short-gap ablations provide greater responsiveness in dynamic settings but compromise execution reliability during phases in which frequent action-chunk merging is unnecessary. This trade-off becomes particularly evident in the full-task evaluation of Task 1 with a dynamic obstacle, which combines stable manipulation phases with dynamic obstacle avoidance. Consequently, all baseline methods achieve full-task success rates of no more than 15%.

In contrast, our method dynamically adapts the inference timing according to the detected scene events. It maintains a long inference gap during stable phases to preserve motion

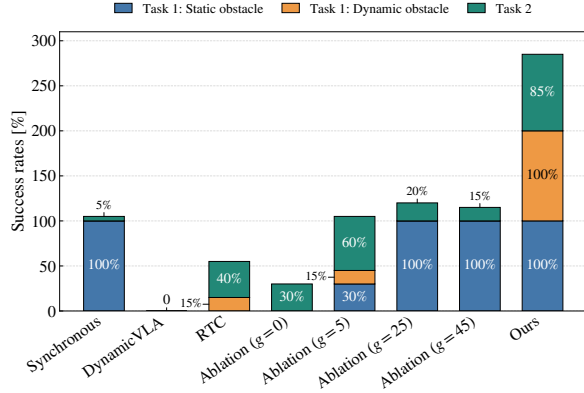


Fig. 8: Aggregate full-task success across three evaluation settings, with each stacked segment representing one setting.

consistency, while promptly triggering new policy inference when significant environmental changes are detected. This adaptive behavior enables reliable execution in static settings while retaining sufficient responsiveness to moving obstacles and targets. Consequently, our method achieves success rates of 100%, 100%, and 85% across the three settings, demonstrating consistently high task performance under both static and dynamic conditions.

VII. CONCLUSION

This paper presented an event-triggered dynamic inference strategy for responsive and reliable asynchronous VLA execution. Unlike existing methods that use a fixed inference gap, our approach adapts inference timing according to scene changes observed since the most recent policy inference. It maintains the largest feasible gap when no significant event is detected and triggers early inference upon event detection. Across real-world settings spanning static and dynamic conditions, our method achieves the highest full-task success rate in every setting, outperforming the strongest baseline by 55 percentage points. These results demonstrate that dynamic inference scheduling can simultaneously preserve motion continuity and enable prompt reactions to environmental changes. Future work will evaluate the proposed strategy across a broader range of manipulation scenarios and VLA backbones.

ACKNOWLEDGMENT

The authors used GPT-5.6 to assist with grammatical and linguistic refinement, as they are non-native English speakers. Additionally, GPT-5.6 was utilized to help format and refine the visual presentation of selected figures. Codex was used to help clean the codebase for its open-source release. All generated suggestions and modifications were reviewed and verified by the authors.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS)*, 2023.
- [2] B. Zitkovich, T. Yu, S. Xu, P. Xu, and X. o. Ted, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning (CoRL)*. PMLR, 2023, pp. 2165–2183.
- [3] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch *et al.*, “PaLM-E: An embodied multimodal language model,” in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 8823–8841.
- [4] O. M. Team, D. Ghosh, H. Walke, D. Shah *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.
- [5] M. J. Kim, L. Karlinsky, B. Prasanna, T. Z. Zhao *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Conference on Robot Learning (CoRL)*, 2024.
- [6] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [8] K. Black, H. Walke, T. Z. Zhao, S. Levine, and C. Finn, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [9] M. Shukor, D. Aubakirova, F. Capuano *et al.*, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” 2025.
- [10] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, and Y. Iwasawa, “Leave no observation behind: Real-time correction for VLA action chunks,” 2025.
- [11] J. Tang, Y. Sun, Y. Zhao *et al.*, “VLASH: Real-time VLAs via future-state-aware asynchronous inference,” 2025.
- [12] K. Black, A. Z. Ren, M. Equi, and S. Levine, “Training-time action conditioning for efficient real-time chunking,” 2025.
- [13] H. Xie, B. Wen, J. Zheng *et al.*, “DynamicVLA: A vision-language-action model for dynamic object manipulation,” *arXiv preprint arXiv:2601.22153*, 2026.
- [14] F. Yang, P. Jing, K. Qu, N. Zhao, and Y. Su, “ABPolicy: Asynchronous B-Spline flow policy for real-time and smooth robotic manipulation,” in *2026 IEEE International Conference on Robotics and Automation (ICRA)*, 2026.
- [15] C. Chi, Z. Xu, S. Feng *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [16] Y. Liu, J. I. Hamid, A. Xie, Y. Lee, M. Du, and C. Finn, “Bidirectional decoding: Improving action chunking via closed-loop resampling,” 2024.
- [17] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess *et al.*, “FAST: Efficient action tokenization for vision-language-action models,” in *arXiv preprint arXiv:2501.09747*, 2025.
- [18] C. Chi, S. Feng, Y. Du *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [19] Y. Guo, Y. Hu, J. Zhang, Y.-J. Wang *et al.*, “Prediction with action: Visual policy learning via joint denoising process,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [20] J. Liu, H. Chen, Z. Liu, P. An *et al.*, “HybridVLA: Collaborative diffusion and autoregression in a unified vision-language-action model,” *arXiv preprint arXiv:2503.10631*, 2025.
- [21] Z. Zhou, Y. Zhu, M. Zhu, J. Wen *et al.*, “ChatVLA: Unified multimodal understanding and robot control with vision-language-action model,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 5376–5390.
- [22] K. Black, N. Brown, D. Driess, A. Esmail *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24158*, 2024.
- [23] K. Black, N. Brown, J. Darpinian, K. Dhabalia *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of The 9th Conference on Robot Learning (CoRL)*. PMLR, 2025, pp. 17–40.
- [24] J. Bjorck, F. Castañeda, N. Cherniadev *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [25] E. J. Hu, Y. Shen, P. Wallis *et al.*, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.